

VisionQueue

Master PRD

Reconciles and supersedes VisionQueue_PRD.pdf and VisionQueue_V1_PRD.pdf. Produced by auditing both for contradictions, gaps, and untestable requirements (Section 17).

Status: Draft for team review — still a PRD, not a Technical Design Document (Section 18).

1. About This Document

This document reconciles the two prior PRDs into one authoritative specification. Where they agreed, that content carries forward unchanged. Where they conflicted, omitted something, or left a requirement untestable, Section 17 records the finding and this document reflects the resolution — conflicts are not silently picked between.

This is still a PRD, not a TDD. It defines what the system must do and pins down precisely, testably, how its behavior must work — not which libraries, frameworks, or architecture implement it. Section 18 lists what's intentionally kept out for that reason, to be picked up in a separate Technical Design Document.

2. Overview

VisionQueue is a real-time computer-vision system that turns a single live camera feed into plain-language queue and crowd information on a live dashboard, without requiring the person watching it to understand computer vision. V1 is the version buildable in roughly 20 days by a small student team, at zero to minimal cost, on a laptop with an optional phone camera.

3. Users and Problem

Primary user: whoever is responsible for a physical space with a queue or crowd — realistically, for V1's pilot, a campus canteen/library queue or one cooperating shop counter (Section 13), not a hospital or airport. Today that person watches a feed continuously, mentally estimates crowd size, and notices a stalled queue only if they happen to be looking. V1's goal: prove one camera can become a live, trustworthy count and crowd-level reading — nothing reported as certain when it isn't, nothing hidden when it breaks.

4. How V1 Actually Runs

Camera input is either the laptop's own webcam, or a phone camera streamed to the laptop (a phone-as-webcam app over WiFi or USB); the phone, if used, is a camera only and does no processing. All computer-vision work — detection, tracking, occupancy/crowd-level calculation, dashboard rendering — runs locally on the laptop. Nothing leaves the laptop by default (FR-20). A phone-over-WiFi source adds latency and depends on WiFi stability; the built-in webcam is the simpler default.

5. Scope: P0 / Core+ / P2 / Advanced

Audit finding (Section 17, #1): the V1-only PRD's single "Later" list had collapsed the fast-follow and deferred tiers into one undifferentiated bucket, losing the original sequencing signal. Restored here as four tiers, used consistently through the rest of this document:

- **P0** — mandatory foundation
- **Core+** — initial-product enhancements, only after P0 is reliable
- **P2** — deferred queue intelligence
- **Advanced** — long-term roadmap (Section 16)

P0 — build now, mandatory foundation

- Live feed from a laptop webcam or phone-as-webcam (Section 4)
- Person detection and face detection running together — face detection satisfies the college requirement; person detection is the actual counting signal
- One configurable ROI; only that zone counts
- Person tracking across frames (consistent IDs) — see Section 6 for the precise rules
- Current count, plus an approximate running unique-people-this-session count
- Entry/exit detection via a configured boundary crossing
- Occupancy % against an operator-set capacity
- A plain crowd-level label (LOW/MODERATE/HIGH/CRITICAL, configurable thresholds, debounced — Section 6)
- One live dashboard: feed, current/unique count, occupancy, crowd level, confidence/camera state
- Exactly two alert types required for P0 — critical occupancy, camera/detection failure — both debounced, built on an extensible mechanism (Section 6) so validated P2 alert types can be added later without redesigning it

- Honest failure behavior across five explicit states, not just "on/off" (Section 6)

Core+ / Initial Product — implement once the P0 foundation is proven reliable

- Crowd trend: INCREASING / STABLE / DECREASING
- Peak crowd count and peak timestamp for the current session
- Basic historical logging (timestamp, count, occupancy, crowd level)
- Basic historical analytics and a simple live/historical crowd-over-time chart
- Basic statistics such as peak crowd and average occupancy

Core+ does not weaken P0: P0 must be complete and reliable before Core+ is considered done (Section 15). These extend the same P0 dashboard — a trend indicator, a peak stat, a simple chart — they don't replace or redesign it. See FR-27–30.

P2 — deferred, not committed to a timeline

- Waiting-time estimation
- Queue movement/stagnation analysis
- Queue abandonment
- Queue jumping
- More sophisticated alert types beyond the two P0 alerts
- Snapshot/event recording

These depend on tracking being proven reliable first (Section 14) and/or need additional validation before they're trustworthy enough to ship — deferred, not rejected.

Advanced — long-term roadmap (Section 16)

- Multi-zone, multi-camera, cross-camera tracking, density heatmaps, gesture assistance, loitering/restricted-area detection, prediction, staffing recommendations, AI operations assistant, voice, cloud/multi-location

6. Precise System Behavior

This section exists because several requirements in the source PRDs were correct in intent but not yet testable as written (Section 17). Product-level decisions below are marked as such, not presented as researched fact; anything requiring a number is marked OPEN DECISION or TBD rather than invented.

Counting and tracking

- **Current count** = number of active tracks whose latest position is inside the ROI, right now
- **Unique people (session)** = count of distinct track IDs that have existed inside the ROI this session — approximate by definition, since ID switches and re-entry can inflate it
- **Session** = from start to stop/restart. Live state (current count, active tracks) does not persist across a restart; logged history (Core+) does
- A track is created when a detected person can't be matched to an existing active track (matching method itself is TDD scope)
- **OPEN DECISION:** how many frames/seconds an unmatched track stays alive before being considered ended — set during testing, not invented here
- A person who re-enters after that timeout gets a new track ID. This inflates the unique count and is an accepted V1 limitation, not a defect (FR-23)
- **Product decision:** boundary crossing is evaluated at the bottom-center point of a person's bounding box (the feet position), and counts at most once per track per crossing (FR-24)
- Current/unique counts are explicitly labeled unreliable and frozen at their last value whenever the system is in UNSTABLE or OFFLINE state (below) — they do not keep silently incrementing

Occupancy and crowd level

- $\text{occupancy \%} = \text{current_count} / \text{configured_capacity} \times 100$
- **Fixes a real gap:** if capacity is 0 or unset, the system must not divide by zero — it shows "capacity not set" instead of a number (FR-22)
- Thresholds (LOW/MODERATE/HIGH/CRITICAL) are configurable percentages, strictly increasing; default starting values are an **OPEN DECISION** to validate at the pilot site, not invented here
- Displayed occupancy rounds to the nearest whole percent
- **Fixes a real gap:** the crowd-level label uses the same debounce window as alerts before changing — a value oscillating right at a threshold must not flicker the label (FR-25). The raw current count itself is *not* debounced; only the derived label and alerts are

Reliability states

Five explicit states, not an implicit on/off:

State	Meaning
STARTING	Initializing camera/model; before the first confident reading exists
LIVE	Normal operation: camera connected, detection confident, performance usable
DEGRADED	Camera connected and detection running, but FPS or confidence has dropped below usable — last reliable count shown, explicitly marked degraded (FR-21)
UNSTABLE	Detection reliability has broken down, not just slowed — last reliable count shown with the unstable indicator
OFFLINE	Camera feed lost entirely — last reliable count preserved, no live detection

Any state can move to OFFLINE if the feed is lost; OFFLINE returns to STARTING (not straight to LIVE) when the feed returns, so a fresh confidence check happens before the dashboard is trusted again. STOPPING is a sixth, operator-initiated state, distinct from any failure.

Alerts (two required for P0, on an extensible mechanism)

- Each alert has: a trigger condition, a debounce window before it fires, a clear condition (the trigger condition becomes false and stays false for the same window), and a timestamp for both fire and clear
- **Fixes a real gap:** an alert cannot re-fire while already active; it must clear first, then re-trigger, before firing again — symmetric hysteresis, not just one-directional debounce
- Both P0 alerts are the same severity in V1 — a multi-severity system is not required and is not added here
- The trigger/debounce/clear/log pattern above is a reusable mechanism, not hard-coded to exactly two conditions — validated additional alert types (P2) can be added later without redesigning it. No additional P0 alerts are added now

A few explicit non-requirements (found during audit, resolved by scoping out, not by building more)

- A moved or bumped camera silently invalidates the ROI — V1 does not detect this; it's a documented known limitation, not a requirement to solve
- An ROI that doesn't match the physical queue area is an operator configuration error — V1 does not validate ROI sanity
- Changing camera, ROI, or capacity mid-session is not required to be seamless — restarting the session is acceptable V1 behavior
- A laptop going to sleep is treated the same as any other feed interruption — no special case is needed beyond the existing OFFLINE handling

7. User Stories

ID	Story	Related
US-1	As an operator, I want to see the live feed with detections overlaid, so I can confirm the system is actually working.	FR-01, 03, 04
US-2	As an operator, I want to set the queue area on the camera view, so people just passing by don't affect my count.	FR-07
US-3	As an operator, I want the current number of people in the queue right now, so I know how busy it is without walking over.	FR-09, 10
US-4	As an operator, I want occupancy shown as a % of a capacity I set, so "busy" means something specific to my space.	FR-12, 22
US-5	As an operator, I want a plain, stable crowd-level label instead of raw numbers, so I can read it at a glance without it flickering.	FR-13, 25
US-6	As an operator, I want to be told when the system isn't confident — or is merely running slow — so I don't trust a bad reading.	FR-06, 17, 21
US-7	As an operator, I want an alert only when occupancy is genuinely critical, so I'm not interrupted by noise.	FR-14
US-8	As an operator, I want the last known state kept if the camera drops, so I'm never staring at a false zero.	FR-02, 17
US-9	As a college evaluator, I want to see faces detected live, so the computer-vision requirement is visibly satisfied.	FR-04

8. Functional Requirements and Acceptance Criteria

FR-01 through FR-20 carry forward from the V1 PRD. FR-21 through FR-26 fix specific audit findings (Section 17). FR-27 through FR-30 are Core+ requirements — not P0, and not required for the 20-day P0 deadline (Section 15).

ID	Requirement	Acceptance criteria
FR-01	Accept a live feed from a laptop webcam or a phone used as a webcam	Feed visible on the dashboard within ~2s of start
FR-02	Detect and report when the camera feed becomes unavailable	Dashboard shows OFFLINE, not a blank or zero reading
FR-03	Detect people within the configured ROI in each processed frame	Multiple people in frame are each detected, not just one
FR-04	Detect faces within the configured ROI in each processed frame	Runs visibly alongside person detection, satisfying the college requirement
FR-05	Attach a confidence value to every detection	Confidence is available per detection, not just a bounding box
FR-06	Exclude detections below a configurable confidence threshold from headline counts	Low-confidence boxes are distinguished and not counted by default
FR-07	Let the operator define one rectangular ROI on the camera view	ROI persists for the session; detections outside it are excluded from queue metrics
FR-08	Assign a tracking ID to each detected person and maintain it across frames	Under normal (non-crowded) conditions, a person keeps one ID as they move
FR-09	Report the current tracked-people count within the ROI	Updates live as people enter/leave; not debounced (Section 6)
FR-10	Report a running unique-people-this-session count, labeled approximate	Count increments only for a genuinely new tracking ID, not every frame
FR-11	Detect entry/exit via a configured boundary crossing	Crossing correctly increments or decrements the current count
FR-12	Let the operator configure a numeric capacity	Occupancy % = current count / capacity, calculated live
FR-13	Classify current occupancy into a plain crowd level	LOW/MODERATE/HIGH/CRITICAL, thresholds configurable, debounced (FR-25)
FR-14	Alert when occupancy stays CRITICAL past a debounce window	No alert on a single-frame spike; alert is logged with a timestamp
FR-15	Alert when the camera feed is lost	Logged with a timestamp; does not spam if the feed flickers
FR-16	Show one dashboard view: feed, current/unique count, occupancy, crowd level, confidence state, camera status	Operator never needs a second screen or a log file to see current state
FR-17	On detection failure, show the last reliable count with an explicit "unstable" indicator	Never displays a fabricated "0"
FR-18	Never silently fail	Any internal error affecting counts is visible to the operator, not hidden
FR-19	Perform no biometric identification of any tracked person	No name, face-print, or persistent cross-session identity is created or stored
FR-20	Process video and run detection/tracking locally on the laptop	No frame, detection, or count data is sent to an external service by default
FR-21	Surface a DEGRADED state when sustained FPS or confidence drops below usable, distinct from UNSTABLE/OFFLINE	Dashboard shows DEGRADED with the last reliable count, not silently laggy
FR-22	Refuse to calculate occupancy when capacity is unset or zero	Dashboard shows "capacity not set", never a divide-by-zero or nonsense value
FR-23	Treat a person re-entering after the track-timeout as a new track	Documented as a known limitation; unique count may over-count accordingly
FR-24	Evaluate crossings at one fixed reference point per person, counted at most once per crossing	Bottom-center of the bounding box; no double-counting from hovering near the line

ID	Requirement	Acceptance criteria
FR-25	Debounce the crowd-level label the same way alerts are debounced	Label only changes after the new value holds for the configured window
FR-26	Start a new session on restart; live state does not persist, logged history (Core+) does	Current/unique counts reset on restart; historical logs from before it are untouched
FR-27	(Core+) Determine crowd trend (increasing/decreasing/stable) from recent count/occupancy history	Trend reflects a rolling window of logged data, not a single reading
FR-28	(Core+) Record peak count, peak occupancy %, and time of peak for the current session	Peak updates only on a new maximum; resets on new session (FR-26)
FR-29	(Core+) Log timestamped count/occupancy/crowd-level data at a configurable interval	Log entries persist across restart (Section 11)
FR-30	(Core+) Display basic historical statistics and a simple crowd-over-time chart	Derived only from FR-29's logged data; empty state if none yet; does not slow the P0 real-time dashboard (Section 9)

9. Non-Functional Requirements

- **Real-time:** the processing loop runs continuously on live video, not recordings; FPS target is set after testing on the team's own laptop, not guessed here
- **Reliability:** recovers from a brief camera disconnect without a manual restart; distinguishes "no people present" (a valid live reading of zero) from any failure state
- **Privacy:** no persistent biometric identity; nothing kept beyond what avoids double-counting within a session (FR-19)
- **Security:** local-only processing by default; no open network service exposed without a specific reason
- **Usability:** someone with no computer-vision background can read the dashboard without being trained on it
- **Maintainability:** detection, tracking, and dashboard are separable pieces of code — this is the direct input to the TDD (Section 18)

10. Privacy, Security, and Legal Context

V1 does not identify anyone by name or store biometric identity (FR-19) — tracked IDs exist only to avoid double-counting within a session and are discarded afterward.

India's Digital Personal Data Protection Act, 2023 treats CCTV footage and biometric data such as face images as personal data, but its core operational provisions are still being phased in; video surveillance today is governed mainly by the IT Act, 2000 and the 2011 IT Rules. Obtaining individual consent for public CCTV is generally considered impractical, and the Act doesn't yet specify a mechanism for it — a genuinely open question this document can't resolve. This is not legal advice.

- Prefer a consented pilot (Section 13): a site owner who can give direct informed consent and post signage sidesteps the harder open question of consent at true public scale
- No raw video retention beyond the live processing buffer; no snapshot storage by default (P2)
- Local processing only; no hard-coded credentials or open, unauthenticated network services, even in a demo build
- V1 local student pilot and any future public/commercial deployment are explicitly different cases — a full legal review is a prerequisite for the latter, not the former

11. Conceptual Data Model

Kept conceptual — column-level schema, indexes, and a specific storage engine are TDD scope (Section 18).

Entity	Purpose	Persists across restart?
Session	One monitoring run, start/end time	No — new session per restart (FR-26)

Entity	Purpose	Persists across restart?
Camera/ROI configuration	Operator's current setup: source, ROI, capacity, thresholds	Yes — it's setup, not live state
Occupancy measurement (Core+)	Timestamped count/occupancy/crowd-level snapshots	Yes, once logged
Alert event (P0)	Timestamp, type, trigger condition, clear time	Yes, once logged
System event	State transitions from Section 6 (e.g. LIVE→DEGRADED) and why	Yes, once logged

None of this requires a database server in V1 — a single local, zero-setup store fits the local/zero-budget constraint (Section 12) better than one that does. No raw video or snapshots are stored by default.

12. Technical Constraints

- Roughly 20 days for this V1
- Zero to minimal budget — no paid AI subscriptions, no paid infrastructure
- Treat the laptop as CPU-only until a GPU is confirmed
- Free/open-source tooling only
- Camera architecture as in Section 4 — laptop does all processing regardless of camera source
- Model licensing: some popular pretrained models carry terms (e.g. AGPL-style) that are fine for a student build but worth a specific check before any future commercial use — not a V1 blocker, revisit in the TDD/Technology Evaluation stage

13. V1 Pilot Environment

An on-campus queue (canteen line, library circulation desk) or one cooperating shop counter — not a hospital, bank, or airport. Accessible without formal institutional approval, and the site owner can give direct, informed consent and post signage.

14. Key Risks

Risk	Why it matters	What to do
Tracking IDs switch when people overlap or briefly leave frame	Directly corrupts the unique-people count and anything built on it	Use a well-documented, lightweight tracker; test specifically with people crossing paths; if unreliable near the deadline, report current count only
CPU-only laptop can't sustain real-time detection + tracking together	A laggy dashboard undercuts the entire "real-time" claim	Smallest/fastest detection model available; lower input resolution; detect every few frames with tracking filling the gaps; surface DEGRADED (FR-21) rather than hide it
Face detector alone misses people not facing the camera	The college requirement is about faces, but the product's real job is an accurate count	Person detection is the counting signal (FR-03); face detection satisfies the requirement, not the count
20 days is tight even for this trimmed, now-more-precise list	Finishing 80% of everything is worse than finishing 100% of less	Build P0 in FR dependency order first; if behind schedule, cut Core+ entirely before any P0 reliability requirement is touched
Undefined thresholds (track-timeout, crowd-level %s) get guessed under time pressure and never revisited	An invented number quietly becomes a de facto requirement	Both are marked OPEN DECISION in Section 6 on purpose — validate them against real pilot footage before the demo, not after

15. V1 Definition of Done

- Live camera works and shows OFFLINE correctly when it doesn't; STARTING is distinct from LIVE
- Person and face detection both run and are visible in the demo
- ROI filtering excludes out-of-zone detections
- Tracking assigns consistent IDs under normal (non-crowded, non-occluded) conditions
- Current count, unique count (approximate), occupancy %, and crowd level are all live on the dashboard
- Entry/exit crossing updates occupancy correctly, counted once per crossing (FR-24)
- Critical-occupancy and camera/detection-failure alerts fire, debounce and clear correctly, and are logged
- DEGRADED, UNSTABLE, and OFFLINE are each visibly distinct states, never a bare zero
- Zero/unset capacity shows "capacity not set", not an error or nonsense value
- Tested against real video of the actual pilot queue, not only synthetic clips
- Known limitations (heavy-occlusion tracking, re-entry creating new IDs, camera-movement not detected, single camera only) are written down, not discovered live during the demo

If Core+ is implemented (not required for the P0 deadline)

- Trend (FR-27) correctly reflects count/occupancy history, not a single reading
- Peak crowd (FR-28) is calculated correctly and resets per session
- Historical records (FR-29) are stored correctly and persist across a restart
- Basic analytics and the chart (FR-30) are derived only from recorded data, with a clear empty state when there's none yet
- Core+ does not measurably slow or interfere with P0's real-time detection/tracking (Section 9)

Core+ is not mandatory. If it threatens the 20-day P0 deadline, it is cut — see Section 14.

16. Advanced Version Scope (V2 and Beyond)

Kept separate from P0/Core+ on purpose. Each stage below has a prerequisite — the roadmap is sequenced by what it actually depends on, not by interest level (Section 17, audit of roadmap coherence).

Version	Focus	Prerequisite before starting
V2	Improved tracking/counting, queue length, further improved waiting-time estimation, queue flow rate, multi-zone, density heatmaps, gesture assistance	P0 tracking proven reliable in real conditions, not just in testing
V3	Loitering, restricted areas, abnormal movement/event detection, multi-camera, cross-camera tracking, stronger analytics	V2's tracking improvements; multi-camera needs a proven single-camera version first, not a redesign
V4	Crowd forecasting, waiting-time prediction, counter/staff recommendations	Real historical data from Core+ logging over a meaningful period — forecasting from no data forecasts nothing
V5	AI operations assistant, natural-language analytics, voice, multi-location, cloud	A stable analytics API for the assistant to call as tools (question → tool call → VisionQueue data → explanation), never inventing answers from raw video. Also gated by budget — this is the one stage that needs an LLM, API or self-hosted

17. Audit: Findings and Resolved Conflicts

From auditing both source PRDs against each other and against the pipeline (camera through dashboard) for untestable or missing behavior. Real findings only — where a question turned up nothing to fix, it's not listed here as padding.

#	Finding	Resolution	Severity
1	The V1-only PRD's "Later" list collapsed the fast-follow and deferred tiers into one bucket, losing the sequencing signal from the comprehensive PRD	Restored four-tier scope: P0/Core+/P2/Advanced (Section 5)	HIGH
2	occupancy = count/capacity was undefined at capacity = 0; neither source PRD addressed it	Explicit "capacity not set" state instead of a divide-by-zero (FR-22)	HIGH
3	Debounce was defined for alerts but not for the crowd-level label, which could flicker at a threshold boundary	Label now uses the same debounce window as alerts (FR-25)	MEDIUM
4	"Line/zone crossing" was never tied to a specific point on a bounding box, leaving FR-011/013 untestable as written	Defined as bottom-center of the box, counted once per crossing (FR-24)	MEDIUM
5	Re-entry after leaving frame was a named limitation in the comprehensive PRD but dropped entirely from the V1-only PRD	Re-added explicitly (FR-23) and in the Definition of Done	MEDIUM
6	Neither PRD defined a state for "running but behind" (dropped frames/low FPS short of full failure)	Added DEGRADED, distinct from UNSTABLE/OFFLINE (Section 6, FR-21)	MEDIUM
7	Alert "debounce" was specified for triggering but not for clearing or re-arming	Symmetric hysteresis: must clear before it can re-fire (Section 6)	LOW
8	No state existed for the first few seconds of initialization, or for an operator-chosen stop	Added STARTING and STOPPING to the state model (Section 6)	LOW
9	Track-loss timeout (how long a lost track stays alive) was never specified, though it drives re-entry/uniqueness behavior directly	Marked OPEN DECISION — set during testing against real footage, not invented here	MEDIUM
10	A moved/bumped camera silently invalidates the ROI with no detection mechanism in either PRD	Logged as a known V1 limitation, not a requirement to build camera-movement detection	LOW
11	Neither PRD defined what happens to live state across an application restart	New session on restart; live state resets, logged history (Core+) doesn't (FR-26)	LOW
12	A literal database schema, API/WebSocket contracts, and a frontend framework decision were requested at PRD level, which repeats the PRD/TDD mixing both source PRDs explicitly avoid elsewhere	Kept conceptual here (Section 11); schema, API, frontend, and full tech-stack comparison moved to the TDD (Section 18)	SCOPE DECISION

18. Explicitly Out of Scope for This Document

Requested alongside this audit, and genuinely useful — but TDD/Technology-Evaluation work, not PRD work. Listed here so nothing is lost, just correctly sequenced as the next document:

- Final technology stack decision (detection model, tracker, inference runtime, backend, frontend framework, storage) with alternatives compared and rejected, separately for V1 and the Advanced version
- Literal database schema (tables, columns, types, indexes)
- API design: REST endpoints, WebSocket contracts, request/response/error schemas, reconnection behavior
- Frontend framework choice and the actual dashboard UI/visual direction
- Module interfaces (Detector, Tracker, CameraSource, Storage, etc.) for future replaceability
- Testing strategy (unit/integration/CV-evaluation/failure-injection tests)
- Day-by-day 20-day build schedule

Detection-model, tracker, and runtime research is already underway and will be evaluated in the TDD against these capabilities — person detection, face detection, tracking, real-time operation on the team's actual hardware, measurable accuracy, and licensing — not committed to a specific model generation here.

19. Open Questions

- Is the pilot site (Section 13) confirmed, or should the team plan for a staged/controlled queue instead?
- Is there a GPU available at all, even a modest one?
- How many of the 20 days are left as of today?
- Track-loss timeout and crowd-level default thresholds (Section 6) — both intentionally left open, to be set from real pilot footage rather than guessed